
Platform for Protecting Water Chlorination Systems from Cyber Attacks

Phase 1 / MVP Progress Report

A Process-Aware OT/ICS Anomaly-Detection and Monitoring System

Team (graduation project): Marko · Mahmoud · Kirolos · Kerolos Yousef

Program: M.S. in Computer Science Supervisor: Dr. Zulkernine

Detector: TranAD (selected & trained) Dataset: SWaT clean Attack_v0

Date: September 12, 2026 Repository HEAD: e57a18d (docs/root-readme)

Motivation & Background

- **OT/ICS run physical processes** — in a water-treatment plant, PLCs read sensors and drive pumps/valves to hold chlorine dosing in a narrow safe band — too little is unsafe to drink, too much is hazardous.
- **A different security model** — OT is engineered primarily around process safety, reliability, and availability; its threat models differ from conventional IT systems.
- **Connectivity increases exposure** — remote access and enterprise integration raise the attack surface; consequences are cyber-physical, not just digital.
- **Perimeter monitoring is necessary but may be insufficient** — an attacker can use legitimate control mechanisms to induce unsafe physical behaviour that looks like valid traffic.
- **Our angle: process-aware detection** — evaluate whether observed multivariate telemetry stays consistent with learned plant behaviour.

Problem Statement & Research Objectives

Problem How can anomalous, cyber-induced changes in multivariate water-treatment process telemetry be detected and surfaced to an operator through a usable monitoring system?

- O1** Analyze the SWaT dataset (normal + labelled attacks) **[ACHIEVED]**
- O2** Integrate & evaluate the selected detector under a deployment-oriented protocol **[ACHIEVED]**
- O3** Deliver a monitoring platform with an operator dashboard **[ACHIEVED (local)]**
- O4** Demonstrate live detection on SWaT attacks (low latency / low FP) **[PARTIAL]**
- O5** Treat detection as a secure product: cloud, containers, CI/CD gates **[FUTURE]**
- O6** Technical documentation + final live attack demonstration **[PARTIAL]**

Related Work

- **ICS / water-treatment security** — authoritative OT guidance (NIST SP 800-82 Rev. 3) and water-sector guidance (CISA/FBI/EPA); the SWaT and WADI iTrust testbeds provide the rare labelled attack data this field depends on.
- **Multivariate, reconstruction-based detection** — unsupervised time-series anomaly detection — USAD (adversarial autoencoders), GDN (graph neural networks), and TranAD (transformer, two decoders, focus score).
- **TranAD — the model productionized here** — reconstructs the final timestep of each window; strong on SWaT/WADI benchmarks while light enough for CPU inference.
- **Rigorous evaluation** — point-adjustment inflates scores (Kim et al.) — even a random detector scores well; so point-wise metrics are treated as primary.
- **ML systems / DevSecOps** — immutable versioned artifacts, reproducible preprocessing, thin service boundaries; security gates in CI/CD deferred to later phases.

Approach — End-to-End MVP Pipeline

SWaT telemetry

licensed clean Attack_v0 CSVs (not committed)

SWaTReplay

ordered 1 Hz stream · subsample 5 → 5 s cadence

RollingWindow

owns the 30-sample ring buffer · stride 1

30 × 45 window

raw telemetry in feature-name order

FastAPI /score

thin, stateless boundary: validate → delegate

TranADDetector

stateless: preprocess → reconstruct → score

Anomaly score + 45-way attribution

which sensors drove the alert

AlertEngine

stateful, in-memory: dedup · OPEN → CLOSED

Monitoring API + Dashboard

/status /alerts /alerts/active · polling UI

Key Architectural Decisions

RollingWindow owns buffering

A deque (maxlen 30) gives automatic stride-1 eviction; validates feature count & finiteness, copies inputs. Imports neither torch nor FastAPI.

TranADDetector is stateless

One (30×45) window in → one decision out. Trivially testable and horizontally scalable; holds no streaming state.

FastAPI is a thin, stateless boundary

Validate shape / feature identity & order → delegate → shape response. No buffering, no duplicated ML logic (backend → ml.src only).

AlertEngine is the only stateful layer

A small state machine: consecutive anomalies → ONE alert (OPEN → dedup → CLOSE). Severity is a labelled heuristic. In-memory only.

The TranAD Detector

- **Input** — a 30-timestep window of 45 process features; model selection is complete — TranAD is not revisited this phase.
- **Cadence & context** — subsampling keeps every 5th 1 Hz sample → 5 s cadence; a 30-sample window $\approx$ 150 s of context and only becomes scoreable once 30 samples accumulate.
- **Why these numbers** — 30 samples are fixed by the transformer's positional encoding; 45 features remain after 6 near-constant columns were dropped in training.
- **Scoring** — reconstruct only the last timestep with two decoders; $\text{score} = 0.5 \cdot \text{MSE}(x_1) + 0.5 \cdot \text{MSE}(x_2)$, averaged over 45 features, vs a fixed threshold.
- **Process-aware evidence** — per-feature error (45-way) shows which sensors drove the alert — at zero extra cost, carried to the operator.
- **Reproducibility** — the five immutable artifacts are byte-identical & SHA-256 verified; golden-vector parity guards the load-bearing transforms.

SWaT Dataset & Provenance Findings

Clean Attack_v0 (reconstructed)

- 449,919 rows · zero missing values · all 45 features populated
- 12.14% row-level attack ratio
- $\approx$ 12.15% windowed (last-timestep) attack prevalence
- 35 contiguous attack label segments
- normal $\rightarrow$ attack $\rightarrow$ normal sequence, merged by timestamp
- licensed (iTrust/SUTD); held locally, not in version control

Provenance findings $\rightarrow$ honest engineering

- **Duplicated normal block** — raw attack ratio deflated to 3.79%
- **Header-spelling mismatch** — 6 model features silently frozen
- **Threshold calibrated on frozen data** — nominal 1% FPR will not hold
- **Only true constant is P301** — frozen features were the data gap

Response: use clean Attack_v0; surface the caveat; reject (not impute) non-finite input; report point-wise metrics at the true base rate.

Stack, Modules & Build Order

Verified stack: Python 3.10.12 · PyTorch 2.9.1 (CPU) · scikit-learn 1.7.2 · NumPy 2.2.6 · joblib 1.5.3 · FastAPI 0.141.1 · Uvicorn 0.52.4 · httpx 0.28.1 · pytest 8.4.2 · vanilla HTML/CSS/JS. No GPU required.

- **ml/** — production TranAD inference + immutable artifacts (research notebooks kept separate).
- **simulator/** — SWaTReplay, RollingWindow, pipeline glue — imports neither torch nor FastAPI.
- **backend/** — thin FastAPI app + request/response schemas.
- **alerting/** — stateful alert engine + Alert value object (duck-types on detector output).
- **frontend/** — single-file zero-build operator dashboard.
- **tests/** — unit · golden-vector ML · API · integration · end-to-end.

Build order: Phase 0 → 1 → 1.5 → 2A → 2B → 3 → 4 → 5A.

2A (replay + rolling window) was built before 2B (FastAPI) on purpose — so buffer ownership was resolved first and the API could stay thin & stateless.

Protocol — Measuring Honestly

- **Point-wise is the primary protocol** — each window is scored against ground truth at its last timestep — the only sample TranAD reconstructs.
- **Point-adjusted is secondary only** — it marks a whole segment detected from a single flagged point and systematically overstates performance (Kim et al.).
- **F1 $\approx$ 0.9999 is invalid as a headline** — it came from a threshold searched on the test set — “can read $\sim$ 1.0 even for a broken model.” Never reported as deployment.
- **Threshold-free metrics reported too** — ROC-AUC and PR-AUC characterize the model independently of the shipped threshold.
- **System-level validation** — 286 automated tests; golden-vector parity; end-to-end over a real Uvicorn socket; normal $\rightarrow$ attack $\rightarrow$ recovery lifecycle.

Preliminary Results



Point-wise, clean Attack_v0, at the shipped threshold 9.269035945180804e-05 — the deployment figures to quote.

Rung	Dataset / labelling / protocol	Prec.	Recall	F1	FPR
1	research concat (defective) / any-in-window / point-adjusted	0.6575	1.0000	0.7934	—
2	clean Attack_v0 / any-in-window / point-adjusted	0.8450	0.9655	0.9012	—
3	clean Attack_v0 / last-timestep / point-adjusted	0.8151	0.9597	0.8815	—
4	clean Attack_v0 / last-timestep / point-wise (deployment)	0.7822	0.7821	0.7822	3.0117%

Rung 4 is the primary deployment metric; rungs 1-3 are secondary point-adjusted diagnostics. Point adjustment does not change FP/TN, so FPR is reported only for the point-wise rung. 286 tests passing.

Achieved Outcomes — Current MVP

- TranAD inference engine (stateless, over immutable artifacts)
- Reproducible, regression-tested preprocessing (golden-vector parity)
- SWaT telemetry replay of the clean Attack_v0 series
- Caller-owned rolling 30-sample window (stride 1)
- Thin, stateless FastAPI backend (/health, /score)
- Stateful in-memory alert engine (dedup + OPEN → CLOSED)
- Monitoring API (/status, /alerts, /alerts/active)
- Zero-build operator dashboard (vanilla HTML/CSS/JS)
- End-to-end demo over a real HTTP boundary (normal/attack/recovery)
- Deployment-oriented evaluation with documented provenance
- 286 automated tests (unit · ML · API · integration · E2E)
- Verified, pinned, reproducible environment

Future Work & Key Risks

Future work (not yet implemented)

- ▶ Alert persistence / database (replace in-memory state)
- ▶ Stuck/frozen-channel detection at ingest
- ▶ Docker containerization
- ▶ CI/CD pipeline + DevSecOps security gates (SAST, dep/container scan)
- ▶ AWS deployment · Terraform IaC · Kubernetes (stretch)
- ▶ Authentication · notifications · observability
- ▶ Performance/latency testing · final hardening
- ▶ Optional approval-gated threshold recalibration (no retraining)

Key risks & mitigations

- **Threshold caveat** → ~3% FPR — disclosed; honest metrics; recalibration deferred (approval-gated)
- **150 s context requirement** — inherent — window must fill before first score
- **Stuck channel passes silently** — non-finite rejected now; stuck-channel check is future work
- **In-memory alert state** — lost on restart; persistence is prioritized future work
- **Dataset licensing** — CSVs local & git-ignored; via env var
- **Artifact / env drift** — SHA-256 pins + golden-vector parity test

Conclusion

Phase 1's contribution is not a new detector — it is a reproducible, honestly evaluated engineering pipeline that turns a validated TranAD detector into an operational, end-to-end local monitoring MVP.

- Stateless detector · RollingWindow owns buffering · thin FastAPI · AlertEngine is the only stateful (in-memory) layer
- Deployment metric: point-wise $F1 = 0.7822$, $FPR = 3.0117\%$ · ROC-AUC 0.9429 · PR-AUC 0.8516
- 286 tests passing · end-to-end normal → attack → recovery over real HTTP · threshold caveats disclosed
- Cloud, containers, CI/CD security gates, and persistence are clearly separated as future work

Thank you — Questions & Discussion

References: NIST SP 800-82 Rev. 3 · CISA/FBI/EPA WWS Guide · SWaT/WADI · TranAD · USAD · GDN · Kim et al. (full list in the report, §12).